Final Exam

Software Engineering

What is it? Software engineering is the application of systematic, disciplined, and measurable approaches to the development, operation, and maintenance of software. It applies engineering principles to software creation.

Why is it necessary?

- Software systems are complex — without structure, they become unmaintainable
- Reduces bugs, cost overruns, and project failures
- Ensures the final product actually meets the client's needs
- Makes large team collaboration possible

Build Models

What is a model? A **virtual representation** of the system you're going to build, like a blueprint before constructing a building.

Why build models?

- Get a clearer picture of *how* to build the real system
- Clarify details and requirements so you build the **right** thing
- Catch design mistakes early (much cheaper to fix on paper than in code)

Traceability — the ability to track requirements through the *entire* development process.

Why traceability matters:

- Better for maintenance
- Tells you which parts of the system are related
- Makes it clear what needs to change when something changes

Software Development Life Cycle (SDLC)

The phases in order, and their key work products:

Phase	Work Product
Requirements Elicitation	Use cases, requirements list

Phase	Work Product
Analysis	Refined use cases, class diagrams, sequence diagrams
High-Level System Design	Architecture diagrams, subsystem breakdown
Detailed Object Design	Detailed class diagrams, contracts
Implementation	Source code
Testing / QA	Test cases, bug reports, test results
Deployment & Maintenance	Deployed system, patches, updates

Models in Requirements Analysis

There are **three types of models**, each answering a different question:

Model	What it captures	UML Diagrams
Functional Model	*What* the system does	Use cases, requirements
Dynamic Model	*When/how* the system behaves over time	State machines, sequence diagrams, activity diagrams
Object Model	*What* the system is made of	Class diagrams, data dictionaries

Think of them as: **what it does → how it behaves → what it's made of.**

Requirements Elicitation

Goal: Figure out what the client actually wants.

Two types of requirements:

- **Functional** — what the system *does* (e.g., "user can log in")
- **Non-functional** — qualities the system must have, across **9 categories** (e.g., performance, security, usability, reliability, etc.)

Scenarios & Use Cases:

- **Scenarios** are concrete examples of how a user interacts with the system
- **Use cases** generalize those scenarios

Use Case concepts to know:

- **High-level vs. Detailed** — high-level is a brief summary; detailed includes full flow, preconditions, postconditions, exceptions
- **Actors** — end users *or* external systems that interact with the system
- **System boundary** — defines what's inside vs. outside the system
- **Actor relationships:** *initiate* (starts the use case) vs. *participate* (is involved but doesn't start it)
- **Use case relationships:**
  - **«include»** — one use case always calls another (required dependency)
  - **«extend»** — one use case optionally extends another (conditional behaviour)
  - **Inheritance** — a child use case inherits from a parent use case
- **UML Use Case Diagrams** + **Use Case Table Descriptions** are the deliverables

---

Analysis

Analysis is about refining what we learned in requirements elicitation and building two key models: the **Dynamic Model** and the **Object Model**.

Dynamic Model

Goal: Show the system's behaviour from the **user's point of view** — how the system changes over time in response to events.

Two main diagrams:

1. Sequence Diagrams

- Show how objects **interact over time** in a specific scenario
- Has a timeline (vertical axis) and objects/actors (horizontal)
- Arrows represent **messages/calls** between objects
- Great for mapping use cases into object interactions

2. State Machine Diagrams

- Show the **states** an object can be in, and what **transitions** move it between states
- Key components:
  - **State** — a condition the object is in (e.g., *Locked*, *Unlocked*)
  - **Transition** — triggered by an event (e.g., *insert coin*)
  - **Guard** — a condition that must be true for a transition to fire
  - **Action** — what happens during a transition

Object Model

Goal: Identify the *things* in the system — what they are, what they know, and what they can do.
Where do you find objects? → From the **use cases** (look for nouns!)

Three Types of Objects:

Type	Role	Example
Entity	Stores persistent data	`User`, `Order`, `Product`
Boundary	Interfaces with actors (UI, APIs)	`LoginScreen`, `PaymentGateway`
Control	Coordinates the flow of a use case	`LoginController`, `CheckoutManager`

A helpful rule of thumb: every use case typically has **one control object**, **one or more boundary objects**, and **several entity objects**.

Attributes & Operations

- **Attributes** — what an object *knows* (its data/fields)
- **Operations** — what an object *does* (its methods)

Relationships Between Classes:

Aggregation (a "has-a" relationship):

- **Shared aggregation** — the part can exist independently of the whole (open diamond `◇`)
  - e.g., a `Professor` can exist without a `Department`
- **Composition** — the part *cannot* exist without the whole (filled diamond `◆`)
  - e.g., a `Room` cannot exist without a `Building`

Inheritance (an "is-a" relationship):

- Child class inherits attributes and operations from a parent
- Arrow points **up** to the parent class

Multiplicity & Directionality:

- **Multiplicity** — how many instances are involved (e.g., `1`, `0..*`, `1..*`)
  - e.g., a `Student` enrolls in `0..*` courses
- **Directionality** — whether a relationship is navigable one-way or both ways (shown with arrows on associations)

Deliverables:

- **UML Class Diagrams** — the visual map of all classes and their relationships
- **Data Dictionary** — a written description of every class, attribute, and operation

Summary

Dynamic model = *behaviour over time* (sequence + state diagrams)
Object model = *structure of the system* (class diagrams + data dictionary)

Architectural & Subsystem Design

This is about making high-level decisions on how to **structure and organize** the system before writing any code.

Key Design Concepts & Terminology

- **Component** — a modular, deployable unit of software (e.g., a login module)
- **Node** — physical hardware that components run on (e.g., a server, a client machine)
- **Subsystem** — a group of related components that together provide a service
- **Interface** — defines *what* a subsystem offers without exposing *how* it works (the contract between subsystems)

Coupling & Cohesion

These are the two most important design quality metrics:

Concept	Definition	Goal
Coupling	How much subsystems *depend on each other*	**Low** coupling — changes in one don't break others
Cohesion	How closely related the responsibilities *within* a subsystem are	**High** cohesion — each subsystem does one thing well

> Think of it this way: a good subsystem is **focused inside** (high cohesion) and **independent outside** (low coupling).

Layers & Partitions

Two ways to decompose a system:

- **Layers** — hierarchical organization where each layer only communicates with the layer directly below it
  - e.g., UI layer → Business Logic layer → Database layer

- **Partitions** — divide the system into independent, parallel subsystems at the same level
  - e.g., Billing subsystem | Inventory subsystem | User subsystem

Services & UML Notation

- A **service** is what a subsystem *provides* to others through its interface
- **UML Component Diagrams** show subsystems and their relationships
- **Ball-and-socket notation:**
  - **Ball (lollipop)** — the subsystem *provides* an interface
  - **Socket (half-circle)** ⌓ — the subsystem *requires* an interface

Design Qualities to Know

Principle	What it means
Design for change	Build so future changes are easy and localized
Separation of concerns	Each part of the system handles one distinct concern
Information hiding	Subsystems hide their internal details from others
High cohesion	Keep related things together
Low coupling	Keep unrelated things apart
KISS	Keep It Simple — don't over-engineer

Architecture Styles

These are common patterns for organizing an entire system:

Style	Key Idea	Example
N-Tier	System split into N horizontal layers	Web app: UI / Business / DB
Client-Server	Clients request services from a central server	Email, web browsing
Peer-to-Peer	Every node is both client and server	BitTorrent, blockchain
Repository	All subsystems share a central data store	Compilers, databases
MVC	Separates Model, View, Controller	Web frameworks (Django, Rails)
Pipe & Filter	Data flows through a chain of processing steps	Unix pipes, compilers
Event-Driven	Components react to events asynchronously	GUIs, message queues

Style	Key Idea	Example
Rule-Based	Logic controlled by rules (if-then)	AI expert systems

Subsystem Design

Once the architecture is defined, you refine each subsystem:

- **High-level → Refined decomposition** — break subsystems into smaller components
- **Mapping subsystems to components** — decide which code units implement which subsystem

Key decisions to make:

Decision	Options
Persistent data	Where/how is data stored long-term?
Global control flow	Procedural / Event-driven / Threaded
Access control	**Static** (set at compile time) vs **Dynamic** (checked at runtime)
Hardware/software mapping	Which components run on which nodes → **UML Deployment Diagrams**

Design Patterns

Design patterns are **reusable solutions to commonly occurring design problems**. You don't invent them — you recognize the problem and apply the known pattern.

There are 3 categories:

Creational Patterns

> *How objects are **created***

1. **Singleton**

- **Problem:** You need exactly **one instance** of a class globally (e.g., a config manager, a logger)
- **Solution:** The class controls its own instantiation and returns the same instance every time
- **Key idea:** Private constructor + static `getInstance()` method

2. **Factory / Abstract Factory**

- **Problem:** You need to create objects but don't want the client to know the exact class being created
- **Solution:** Delegate object creation to a **factory class**
- **Key idea:** Decouples the creation logic from the rest of the code

3. **Prototype**

- **Problem:** Creating a new object from scratch is expensive
- **Solution: Clone** an existing object instead
- **Key idea:** `clone()` method on objects

Structural Patterns

> *How objects are **composed/organized***

1. **Adapter**

- **Problem:** Two incompatible interfaces need to work together (e.g., legacy code + new system)
- **Solution:** A wrapper class that **translates** one interface into another
- **Analogy:** A power plug adapter — same electricity, different socket

2. **Facade**

- **Problem:** A subsystem is complex with many classes; clients shouldn't need to know all of it
- **Solution:** Provide a **single simplified interface** to the subsystem
- **Analogy:** A TV remote — hides all the internal complexity

3. **Composite**

- **Problem:** You need to treat **individual objects and groups of objects** the same way
- **Solution:** A tree structure where both leaves and branches share the same interface
- **Example:** A file system — a `File` and a `Folder` are both treated as `FileSystemItem`

4. **Decorator**

- **Problem:** You want to add behaviour to an object **dynamically** without changing its class
- **Solution:** Wrap the object in a decorator that adds the new behaviour
- **Example:** Adding scrollbars or borders to a UI window at runtime

5. **Proxy**

- **Problem:** You need to control access to an object (e.g., lazy loading, security checks)
- **Solution:** A **stand-in object** that controls access to the real object
- **Types:** Virtual proxy (lazy init), Protection proxy (access control), Remote proxy (network object)

Behavioural Patterns

> *How objects **communicate and distribute responsibility***

1. **Strategy**

- **Problem:** You have multiple algorithms for the same task and want to switch between them
- **Solution:** Define a family of algorithms, encapsulate each one, make them interchangeable
- **Example:** Different sorting algorithms, different payment methods
- **Key idea:** Composition over inheritance — pass in the behaviour

2. **Observer**

- **Problem:** When one object changes, many others need to be notified automatically
- **Solution:** Objects **subscribe** to a subject; when it changes, all subscribers are notified
- **Example:** Event listeners, MVC (View observes Model)
- **Key idea:** Subject holds a list of observers and calls `update()` on all of them

3. **Command**

- **Problem:** You want to encapsulate a request as an object (for undo, queuing, logging)
- **Solution:** Wrap each action in a **Command object** with an `execute()` method
- **Example:** Undo/redo in a text editor

4. **Template Method**

- **Problem:** Multiple classes share the same algorithm structure but differ in some steps
- **Solution:** Define the skeleton of the algorithm in a base class; let subclasses fill in the steps
- **Key idea:** "Don't call us, we'll call you" — the parent controls the flow

5. **Iterator**

- **Problem:** You need to traverse a collection without exposing its internal structure
- **Solution:** Provide a standard `hasNext()` / `next()` interface
- **Example:** Java's `Iterator`, Python's `for` loops

6. **State**

- **Problem:** An object's behaviour changes depending on its internal state
- **Solution:** Represent each state as a separate class; delegate behaviour to the current state object
- **Difference from Strategy:** State transitions happen *automatically*; Strategy is chosen *externally*

Quick Pattern Cheat Sheet

Pattern	Category	One-liner
Singleton	Creational	One instance only
Factory	Creational	Delegate object creation
Prototype	Creational	Clone instead of create
Adapter	Structural	Make incompatible interfaces work
Facade	Structural	Simplify a complex subsystem
Composite	Structural	Treat individuals and groups the same
Decorator	Structural	Add behaviour dynamically
Proxy	Structural	Control access to an object
Strategy	Behavioural	Swap algorithms at runtime
Observer	Behavioural	Notify many on one change
Command	Behavioural	Encapsulate requests as objects
Template Method	Behavioural	Fix the algorithm, vary the steps
Iterator	Behavioural	Traverse without exposing internals
State	Behavioural	Behaviour changes with state

Detailed Object Design

This phase is about **bridging the gap** between your analysis models and actual implementation. You're refining class designs to be precise and code-ready.

Application vs. Solution Domain

Domain	What it is	Example
Application Domain	The real-world problem you're solving	"Students enroll in courses"
Solution Domain	The technical world of software & tools	`EnrollmentController`, `ArrayList`, `HashMap`

- During analysis you focus on the **application domain**
- During object design you introduce **solution domain** objects to actually implement it
- Example: you might add a `Database` class or a `Cache` class that has nothing to do with the real world, but is needed for the solution

Types of Inheritance

Specification Inheritance

- Child class is a **true subtype** of the parent
- The child *is-a* parent — it satisfies everything the parent promises
- Example: `Dog` extends `Animal` — a Dog truly is an Animal
- This is what inheritance is **meant** for

Implementation Inheritance

- Child class inherits from parent **just to reuse code**, not because of a true is-a relationship
- Can lead to fragile, confusing designs
- Example: inheriting from `ArrayList` just to get list methods, even though your class isn't really a list
- **Delegation is usually preferred here** instead

Delegation

- Instead of inheriting to reuse behaviour, you **hold a reference** to another object and call its methods
- **"Has-a" instead of "is-a"**
- Much more flexible — you can swap out the delegated object at runtime

```
// Inheritance (tight coupling)
class MyStack extends ArrayList { ... }

// Delegation (looser coupling)
class MyStack {
```

```
    private ArrayList list;  // delegate to this
}
```

> Rule of thumb: **if in doubt, delegate instead of inherit**

Liskov Substitution Principle (LSP)

> *"A subclass should be substitutable for its parent class without breaking the program."*

- If class `B` extends class `A`, anywhere you use an `A` you should be able to use a `B` and everything still works correctly
- Violations of LSP are a sign you're using **implementation inheritance** when you shouldn't be

Classic violation example:

- `Square` extends `Rectangle`
- A `Rectangle` lets you set width and height independently
- A `Square` can't — setting width must also set height
- So `Square` is **not** a true substitute for `Rectangle` → LSP violation!

Contracts

Contracts formally define what a method **promises and expects**. There are three parts:

Invariant

- A condition that is **always true** for an object, no matter what
- Must hold before AND after every method call
- Example: `balance >= 0` for a `BankAccount` — the balance can never go negative

Precondition

- What must be **true before** a method is called (the caller's responsibility)
- If precondition is violated → it's the **caller's fault**
- Example: `withdraw(amount)` requires `amount > 0` and `amount <= balance`

Postcondition

- What is **guaranteed to be true after** a method completes (the method's responsibility)
- If postcondition is violated → it's the **method's fault**
- Example: after `withdraw(amount)`, `newBalance == oldBalance - amount`

How they relate to inheritance:

- **Preconditions** can only be **weakened** (relaxed) in subclasses — you can't add new requirements on the caller
- **Postconditions** can only be **strengthened** (more guarantees) in subclasses — you can't promise less
- **Invariants** must be **maintained** in subclasses

Quick Summary

Concept	Key Point
Application domain	Real-world problem space
Solution domain	Technical/software space
Specification inheritance	True is-a relationship
Implementation inheritance	Reusing code only
Delegation	Has-a, prefer over implementation inheritance
LSP	Subclass must work wherever parent is used
Invariant	Always true for the object
Precondition	Must be true *before* the method (caller's job)
Postcondition	Must be true *after* the method (method's job)

Software Management & DevOps

This section covers how software projects are **organized, tracked, and delivered** over time.

Software Development Process Models

These define the **order and structure** of SDLC activities.

Sequential Models

> Do each phase once, in order — no going back

Waterfall

- Phases flow strictly downward: Requirements → Analysis → Design → Implementation → Testing → Deployment
- **Pros:** Simple, well-documented, easy to manage
- **Cons:** Very rigid — if requirements change, you're in trouble

- **Best for:** Well-understood, stable requirements

V-Model

- Extension of Waterfall — each development phase is **paired with a testing phase**
- Left side goes down (development), right side goes up (testing/validation)

Iterative Models

> Build the system in **repeated cycles**, improving each time

Prototyping

- Build a quick, rough prototype to clarify requirements with the client
- Throw it away (or evolve it) once requirements are confirmed
- **Best for:** When requirements are unclear

Spiral

- Combines iterative development with **risk analysis** at every cycle
- Each loop: Plan → Risk Analysis → Build → Evaluate
- **Best for:** Large, high-risk projects

Unified Process (UP)

- Iterative and use-case driven, organized into 4 phases:
  - **Inception** — define scope and feasibility
  - **Elaboration** — refine requirements, architecture
  - **Construction** — build the system iteratively
  - **Transition** — deploy to users
- All SDLC activities happen in every phase, just with different emphasis

Agile

- Lightweight, flexible, people-focused approach
- Key principles:
  - Working software over comprehensive documentation
  - Customer collaboration over contract negotiation
  - Responding to change over following a plan
  - Individuals and interactions over processes and tools
- Deliver in short **sprints** (1–4 weeks), get feedback, repeat
- Examples: **Scrum**, **Kanban**, **XP (Extreme Programming)**

Software Configuration Management (SCM)

SCM is about **tracking and controlling changes** to software over time so nothing gets lost and everything stays consistent.

The 4 SCM Pillars:

Pillar	What it means
Configuration Identification	Identifying and labelling all items being managed (code, docs, configs)
Configuration Control	Managing and approving changes formally
Configuration Status Accounting	Recording and reporting the status of all items
Configuration Auditing	Verifying that the system matches its documented state

Key SCM Concepts:

- **Repository** — central storage for all project artifacts (code, docs, configs)
  - Example: Git, SVN
- **Baseline** — a **snapshot** of the project at a specific point in time that has been formally reviewed and approved
  - Think of it as a "version stamp" — you can always go back to it
- **Change Control** — the formal process for requesting, reviewing, approving, and implementing changes
  - Prevents uncontrolled/random changes from breaking the system

DevOps

DevOps bridges the gap between **development** and **operations** — the goal is to deliver software **faster and more reliably**.

Continuous Integration (CI)

- Developers **frequently merge** code into a shared repository (multiple times a day)
- Every merge triggers an **automated build and test**
- **Goal:** Catch integration bugs early, keep the codebase always working
- Example tools: Jenkins, GitHub Actions

Continuous Delivery (CD)

- Every passing build is **automatically prepared** for release to a staging environment
- A human still makes the final call to deploy to production
- **Goal:** Always have a deployable version ready

Continuous Deployment

- Goes one step further — every passing build is **automatically deployed to production** with no human intervention
- **Goal:** Maximum automation, near-instant delivery of changes

CI/CD Pipeline Summary:

```
Code commit
    ↓
Automated Build        ← Continuous Integration
    ↓
Automated Tests        ← Continuous Integration
    ↓
Deploy to Staging      ← Continuous Delivery
    ↓
Deploy to Production   ← Continuous Deployment (automated)
                       ← Continuous Delivery (manual approval)
```

Quick Summary

Concept	Key Point
Waterfall	Sequential, rigid, one pass
V-Model	Waterfall + paired testing phases
Prototyping	Build rough draft to clarify requirements
Spiral	Iterative + risk analysis each cycle
Unified Process	Iterative, 4 phases, use-case driven
Agile	Flexible, sprint-based, people-focused
SCM Repository	Central storage for all artifacts
Baseline	Approved snapshot of the project
Change Control	Formal process for approving changes
CI	Frequent merges + automated tests
Continuous Delivery	Always ready to deploy (human approves)
Continuous Deployment	Always auto-deployed to production

Implementation & Mappings

This section is about **translating your object design into actual code and storage schemas**.

Model Transformations & Optimizing the Object Model

Before coding, you may need to **refine your class diagrams** to be more implementation-friendly:

- **Model transformation** — modifying the object model to improve it without changing its behaviour
- Common optimizations:
  - Caching frequently computed values as attributes
  - Splitting complex classes into smaller ones
  - Collapsing trivial classes into attributes of another class
  - Adding solution-domain classes (e.g., data structures, controllers)

Mapping Associations to Collections (in Code)

One-to-One

- Each object holds a **direct reference** to the other

```
class Student {
    IDCard card;  // one student has one ID card
}
```

One-to-Many

- The "one" side holds a **collection** of references to the "many" side

```
class Professor {
    List<Student> students;  // one professor has many students
}
```

Many-to-Many

- **Both sides** hold collections, OR you introduce an **association class**

```
class Student {
    List<Course> courses;
}
```

```
class Course {
    List<Student> students;
}
```

Qualified Associations

- A **qualifier** (key/identifier) is used to narrow a many relationship down to one
- Mapped using a **HashMap/Dictionary** where the qualifier is the key

```
class Course {
    HashMap<String, Student> students; // studentID → Student
}
```

Association Classes

- When an association itself has attributes, you need an **association class**
- Becomes its own class with references to both sides
- Example: `Enrollment` between `Student` and `Course` with attributes like `grade`, `date`

```
class Enrollment {
    Student student;
    Course course;
    String grade;
    Date enrollmentDate;
}
```

Mapping Associations to Storage Schema (Database)

Buried Association

- For **one-to-one** or **one-to-many** — store the foreign key directly in one of the tables
- The foreign key is "buried" inside one table

Association Tables

- For **many-to-many** — create a **separate table** to represent the association
- The association table holds foreign keys to both sides

Mapping Inheritance to Storage

There are **two strategies**:

1. **Vertical Mapping (One table per class)**

- Each class in the hierarchy gets its **own table**
- Child tables store only their **own new attributes** + a foreign key to the parent table
- To get a full object, you **JOIN** tables

Example:

```
Person table:       personID | name | age
Student table:      personID (FK) | major | GPA
Professor table:    personID (FK) | department | salary
```

- No redundancy, clean structure
- Requires JOINs — slower queries

2. **Horizontal Mapping (One table per concrete class)**

- Each **concrete (non-abstract) class** gets its own table with **ALL attributes** including inherited ones
- No joins needed — everything is in one table

Example:

```
Student table:      studentID | name | age | major | GPA
Professor table:    professorID | name | age | department | salary
```

- Fast queries — no JOINs needed
- Redundancy — shared attributes duplicated across tables
- If parent class changes, all tables must change

Summary

Concept	Key Point
One-to-one mapping	Direct reference field
One-to-many mapping	Collection on the "one" side
Many-to-many mapping	Collections on both sides or association class
Qualified association	HashMap with qualifier as key
Association class	Own class when association has attributes
Buried association	Foreign key stored inside a table
Association table	Separate table for many-to-many
Vertical mapping	One table per class, uses JOINs

Concept	Key Point
Horizontal mapping	One table per concrete class, duplicates data

---

Quality Control & Testing

Testing ensures the system **works correctly and meets requirements**. Know the terminology and the differences between techniques cold.

Key Terminology

Term	Definition
Test Case	A specific input + expected output used to test a behaviour
Test Component	The unit of software being tested
Test Stub	A **fake implementation** of a component that the tested component *calls* (replaces a dependency below)
Test Driver	A **fake caller** that invokes the component being tested (replaces the layer above)

Think of it this way: **stubs go below** (fake what you depend on), **drivers go above** (fake what calls you)

Blackbox vs. Whitebox Testing

	Black	White
What you see	Only inputs & outputs	Internal code/logic
Based on	Requirements/spec	Implementation
Who does it	Testers (no code knowledge needed)	Developers
Goal	Does it do the right thing?	Does it do it correctly inside?
Examples	Equivalence testing, boundary testing	Path testing

Test-Driven Development (TDD)

- Write the **test first**, then write the code to make it pass
- Cycle: **Red → Green → Refactor**
  - **Red** — write a failing test
  - **Green** — write just enough code to pass it

    - **Refactor** — clean up the code without breaking the test
- **Benefits:** Forces clear requirements, keeps code testable, catches bugs early

Unit Testing

Testing **individual components** in isolation.

Path Testing (Whitebox)

- Test every possible **execution path** through the code
- Draw a **control flow graph** and ensure every branch is covered
- Coverage criteria:
    - **Statement coverage** — every line executed at least once
    - **Branch coverage** — every true/false branch taken at least once
    - **Path coverage** — every unique path through the code

Equivalence Testing (Blackbox)

- Divide inputs into **equivalence classes** — groups where all values should behave the same
- Test one value from each class (no need to test every value)
- Example: for input age `0–17` (minor), `18–64` (adult), `65+` (senior) — test one from each

Boundary Testing (Blackbox)

- Test values at the **edges** of equivalence classes — bugs cluster at boundaries
- Example: for `18–64`, test `17`, `18`, `64`, `65`
- Always test: **just below, at, and just above** each boundary

State Testing

- Test all **states and transitions** of an object
- Ensure every state is reachable and every transition works correctly
- Based on the **state machine diagram**

Polymorphism Testing

- When a method is **overridden** in subclasses, test each subclass version separately
- You can't assume the parent's test covers child behaviour

Integration Testing

Testing how **components work together**. The order in which you combine them matters.

Big Bang

- Combine **everything at once** and test
- Simple, fast to set up
- When something breaks, very hard to find where
- Need all components ready before you start

Top-Down

- Start with the **top-level components** first, work downward
- Use **stubs** to fake lower components not yet built
- Tests high-level logic early
- Low-level components tested late

Bottom-Up

- Start with the **lowest-level components** first, work upward
- Use **drivers** to fake higher components not yet built
- Tests core/utility components early
- High-level behaviour tested late

Sandwich

- Combines top-down and bottom-up simultaneously
- Work from both ends toward the **middle**
- Faster than pure top-down or bottom-up
- Middle layer gets less thorough testing

Modified Sandwich

- Like sandwich but the **middle layer is also tested individually** before integration
- More thorough than regular sandwich
- Most balanced approach overall

Integration Strategies at a Glance

Strategy	Direction	Uses Stubs?	Uses Drivers?
Big Bang	All at once	❌	❌
Top-Down	Top → Bottom	✅	❌
Bottom-Up	Bottom → Top	❌	✅
Sandwich	Both directions	✅	✅

Strategy	Direction	Uses Stubs?	Uses Drivers?
Modified Sandwich	Both + middle isolated	✅	✅

System Testing

Testing the **entire system** as a whole against requirements.

Functional Testing

- Does the system do **what it's supposed to do**?
- Based on the **functional requirements** and use cases
- Blackbox — testers don't care about internal code

Performance Testing

- Does the system perform **well enough** under load?
- Types:
  - **Load testing** — normal expected load
  - **Stress testing** — beyond normal load until it breaks
  - **Response time testing** — how fast does it respond?

Acceptance Testing

- Does the system meet the **client's expectations**?
- Performed by the **client/end users**, not developers
- Two types:
  - **Alpha testing** — done internally (client tests at developer's site)
  - **Beta testing** — done externally (real users test in real environment)

Summary

Concept	Key Point
Test stub	Fakes a dependency *below* the tested component
Test driver	Fakes the caller *above* the tested component
Blackbox	Test based on requirements, no code visibility
Whitebox	Test based on internal logic/paths
TDD	Write test first → Red → Green → Refactor
Path testing	Cover every execution path (whitebox)
Equivalence testing	One test per input class (blackbox)

Concept	Key Point
Boundary testing	Test at edges of input classes (blackbox)
Big Bang	All at once — hard to debug
Top-Down	High → Low, uses stubs
Bottom-Up	Low → High, uses drivers
Sandwich	Both directions simultaneously
Modified Sandwich	Sandwich + middle tested in isolation
Functional testing	Does it do the right thing?
Performance testing	Does it do it fast enough?
Acceptance testing	Does the client approve?

Ethics

Ethics in software engineering is about **professional responsibility**, understanding that the software you build affects real people.

Professionalism

What is a profession? A vocation requiring:

- A high level of **education**
- Practical **experience**

We trust professionals (doctors, lawyers) to correctly assess problems and act in their clients' best interest.

Attributes of a Mature Profession (the staircase diagram):

```
Education → Accreditation → Skills Development →
Licensing → Certification → Professional Society
→ Professional Development → Code of Ethics
```

Why software engineering is different:

- Certification and licensing are **not required**
- A university degree is **not required**
- Apprenticeship is **not required**
- Professional society membership is **optional**
- No requirements for continuing education

- Most programmers work as part of teams

> This is what makes software engineering an **immature profession** compared to medicine or law, yet the consequences of failure can be just as serious.

Ability to Harm the Public

Despite lacking formal requirements, software engineers carry serious responsibilities:

- Some develop **safety-critical systems** where failures kill people
- Millions rely on software **instead of professionals** (e.g., tax software instead of accountants, diagnostic software instead of doctors)
- Millions rely on **system admins** to keep their information secure

Notable Software Disasters:
Therac-25

- Radiation therapy machine built by AECL
- Hardware safety features were **replaced with software**
- Code was **reused** from earlier models (Therac-6 and Therac-20)
- **Root cause:** Two **race conditions**, concurrent tasks accessing shared variables in the wrong order
  - One in the command screen editing
  - One in the movement of the electron beam gun
- **Results:** 6 accidents, **3 fatalities**, 2 investigations

Lessons learned:

- Concurrent programs are very hard to debug
- Design must be **as simple as possible**
- **Documentation** is crucial
- Code reuse does **not** always mean higher quality
- Systems must be designed to be **fail-safe**

Ariane 5

- 1996 satellite launch vehicle — self-destructed **40 seconds** into its maiden flight
- **$500 million** in uninsured satellites lost
- **Root cause:** A float value was assigned to an integer variable → raised an unhandled exception → system crashed
- Code was **reused from Ariane 4**, a slower rocket with smaller values where the exception was impossible

- **Lesson:** Code reuse across different contexts is dangerous without re-validation

CrowdStrike (2024)

- Faulty software update sent to millions of Windows systems
- A **logic error** caused the Falcon security sensor to crash → systems reverted to **Blue Screen of Death (BSOD)**
- Affected: commercial flights, hospitals, financial services
- **$5.4 billion** in estimated losses
- **Root cause:** Mismatch between provided and expected data — 20 inputs provided but 21 expected → accessing the out-of-bounds value caused a **segmentation fault**
- **Not caught during testing** — test cases never tried to read the 21st value
- **Lesson:** Test cases must cover edge cases and boundary conditions

Software Reliability — Types of Failures:

- **Data entry/retrieval errors** — wrong data entered, or data misinterpreted
  - Example: Florida 2000 election — thousands of voters incorrectly flagged as felons in the database, potentially affecting the election outcome
- **Malfunction of embedded systems** — hardware-integrated software behaving incorrectly
- **Effects range from:** inconvenience → bad business decisions → **fatalities**

ACM Code of Ethics

The **ACM Software Engineering Code of Ethics and Professional Practice** has 8 principles:

Principle	Core Idea
1. Public	Act in the **public's best interest** — safety always comes first
2. Client & Employer	Act in their best interest, but **not at the expense of the public**
3. Product	Ensure products meet the **highest professional standards**
4. Judgment	Maintain **integrity and independence** in your decisions
5. Management	Promote **ethical management** of software development
6. Profession	Advance the **integrity and reputation** of the profession
7. Colleagues	Be **fair and supportive** to colleagues
8. Self	Commit to **lifelong learning** and ethical practice

> Public welfare always takes priority — even over your client or employer.

Making Ethical Decisions — Two Phases:

Phase 1: Brainstorming

- **Identify stakeholders** — everyone involved directly or indirectly
- For each stakeholder, identify:
  - **Risks, benefits, consequences, costs**
  - **Rights**
- **Identify all possible courses of action**, including:
  - I-win-you-lose
  - You-win-I-lose
  - **We-both-win** ← Stephen Covey's Habit #4 — always look for this option first

Phase 2: Analysis

- Identify the **impact** of each course of action on stakeholders
- Consider the **ACM Code of Ethics** and your own morals/experience
- Classify each action as:

Classification	Meaning
Ethically Obligatory	You *must* do this
Ethically Prohibited	You *must not* do this
Ethically Acceptable	Permissible — reasonable people may disagree

- Choose the **best option** based on ethical merits

Case Studies

Case 1: Protecting Personal Data at a Community Clinic

- Three clinic sites, one being a **shelter for battered women and children**
- Director wants computerized records (names, forwarding addresses), networking between sites, web/email access, and staff laptops
- Director is **reluctant to pay for security features** due to small budget

Ethical issues to consider:

- The shelter's clients face **serious physical danger** if their location is exposed
- The engineer has a responsibility to the **public** (Principle 1) — not just the client
- Skimping on security here could be **ethically prohibited**

- Recommending proper security despite budget pushback would likely be **ethically obligatory**

Case 2: Kickbacks and Disclosure

- You're a university programmer evaluating security software packages for student recommendations
- One company offers you: a dinner, conference expenses, and a **percentage of every sale** to the university

Ethical issues to consider:

- These incentives compromise your **independence and integrity** (Principle 4 — Judgment)
- Recommending that company without disclosure would be **ethically prohibited**
- Disclosing the conflict of interest and recusing yourself would likely be **ethically obligatory**
- Accepting gifts that bias your recommendation harms students (public interest — Principle 1)

Summary

Concept	Key Point
Mature profession	Has education, licensing, certification, code of ethics
Software engineering	Lacks formal requirements but carries serious responsibility
Therac-25	Race conditions, code reuse → 3 deaths
Ariane 5	Unhandled integer overflow from reused code → $500M loss
CrowdStrike	Out-of-bounds data + missed test case → $5.4B loss
ACM Principle 1	Public safety always first
Brainstorming phase	Identify stakeholders, risks, rights, courses of action
Analysis phase	Classify actions as obligatory / prohibited / acceptable
We-both-win	Always look for the third option (Covey Habit #4)